

测试数据及结果

Tiny语言的测试

语法树修正代码

```
#include "../grammarAnalyzer/analyzer/analyzer.h"
#include <set>
#include <algorithm>
namespace std
{
    set<string> ignoreSigns = {"space", "tab", "enter", "(", ")", ";", "[", ""]};
    set<string> ops = {"+", "-", "*", "/", ">", "<", "^", "%", "<=", ">=", "=", "<>"};
}
namespace std{
    extern "C" {
        void fixTree(Tree * tree){
            if (tree->children.empty()){
                return;
            }
            for (auto child : tree->children){
                fixTree(child);
            }
            vector<Tree*> newChildren;
            // 对特定符号进行处理
            // exp
            if (tree->children.size() == 3 && ops.find(tree->children[1]->sign) != ops.end()){
                tree->value = tree->children[1]->value;
                tree->children[1]->value = "";
            }
            // if
            if (tree->sign == "if-stmt"){
                tree->value = tree->children[0]->value;
                tree->children[0]->value = "";
                tree->children[2]->children = tree->children[3]->children;
                tree->children[3]->value = "";
                if (tree->children[4]->sign == "else"){
                    tree->children[4]->children = tree->children[6]->children;
                    tree->children[5]->value = "";
                }
            }
        }
    }
}
```

```

    }}
    // repeat
    if (tree->sign == "repeat-stmt"){
        tree->value = tree->children[0]->value;
        tree->children[0]->value = "";
        tree->children[2]->children = tree->children[3]-
>children;

        tree->children[3]->value = "";
    }
    // read
    if (tree->sign == "read-stmt"){
        tree->value = tree->children[0]->value;
        tree->children[0]->value = "";
    }
    // write
    if (tree->sign == "write-stmt"){
        tree->value = tree->children[0]->value;
        tree->children[0]->value = "";
    }
    // assign
    if (tree->sign == "assign-stmt"){
        tree->value = tree->children[0]->value;
        tree->children[0]->value = "";
        tree->children[1]->value = "";
    }
    // 删除应该被忽略的符号

    for (auto child : tree->children){
        if (ignoreSigns.find(child->sign) == ignoreSigns.end()){
            newChildren.push_back(child);
        }
    }
    tree->children = newChildren;

    // 将只有一个子节点且节点值为空的符号合并
    if (tree->children.size() == 1 && tree->value == ""){
        tree->sign = tree->children[0]->sign;
        tree->value = tree->children[0]->value;
        tree->children = tree->children[0]->children;
    }
    // 删除value为空的符号
    newChildren.clear();
    for (auto child : tree->children){
        if (!(child->value == "")){
            newChildren.push_back(child);
        }else{
            for (auto c : child->children){
                newChildren.push_back(c);
            }
        }
    }

```

```

    }
    tree->children = newChildren;

}}}

```

词法分析器测试

- 测试说明
 - 所有token都在源程序中出现
- 测试用源程序

```

{
    test tokens
}
read x; { input an integer }
if 0 < x then { don't compute if x <= 0 }
    fact := 1;
    repeat
        fact := fact * x;
        x := x - 1
    until x = 0;
    write fact { output factorial of x }
else
    write 0 { output 0 if x <= 0 }
end;

{ Additional statements to include all signs }
y := 10 + 5; { addition }
z := y - 3; { subtraction }
w := z * 2; { multiplication }
v := w / 4; { division }
u := v % 3; { modulus }
t := u ^ 2; { exponentiation }
s := t < 10; { less than }
r := s <> true; { not equal }
q := r <= false; { less than or equal }
p := q >= true; { greater than or equal }
o := p > false; { greater than }
n := o = true; { equal }
m := n { assignment }

```

- 测试结果

- ```

1006 read
10 x

```

```
512 ;
1000 if
20 0
508 <
10 x
1001 then
10 fact
513 :=
20 1
512 ;
1004 repeat
10 fact
513 :=
10 fact
502 *
10 x
512 ;
10 x
513 :=
10 x
501 -
20 1
1005 until
10 x
511 =
20 0
512 ;
1007 write
10 fact
1002 else
1007 write
20 0
1003 end
512 ;
10 y
513 :=
20 10
500 +
20 5
512 ;
10 z
513 :=
10 y
501 -
20 3
512 ;
10 w
513 :=
10 z
```

```
502 *
20 2
512 ;
10 v
513 :=
10 w
503 /
20 4
512 ;
10 u
513 :=
10 v
504 %
20 3
512 ;
10 t
513 :=
10 u
505 ^
20 2
512 ;
10 s
513 :=
10 t
508 <
20 10
512 ;
10 r
513 :=
10 s
506 <>
10 true
512 ;
10 q
513 :=
10 r
507 <=
10 false
512 ;
10 p
513 :=
10 q
509 >=
10 true
512 ;
10 o
513 :=
10 p
510 >
```

```

10 false
512 ;
10 n
513 :=
10 o
511 =
10 true
512 ;
10 m
513 :=
10 n

```

•

## 控制语句语法树的生成

### 1. repeat-until

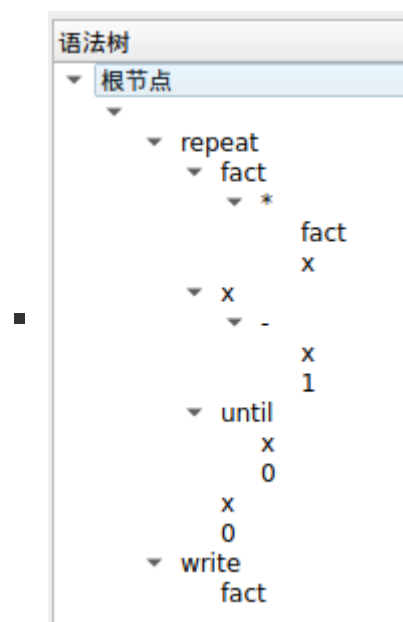
- 测试说明
  - repeat-unti测试
- 测试用源程序

```

repeat
 fact := fact * x;
 x := x - 1
until x = 0;
write fact { output factorial of x }

```

- 测试结果



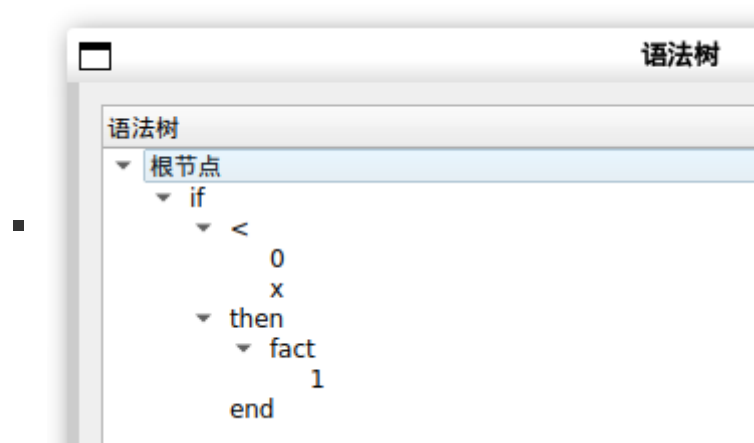
### 2. if

- 测试说明
  - if测试

- 测试用源程序

- ```
{
    Test program if
}
if 0 < x then { don't compute if x <= 0 }
    fact := 1
end
```

- 测试结果



3. if-else

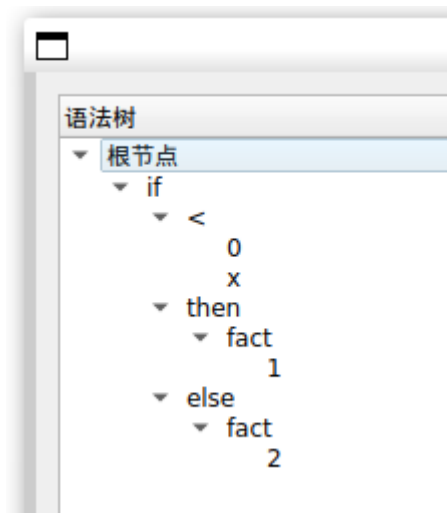
- 测试说明

- if-else测试

- 测试用源程序

- ```
{
 Test program if
}
if 0 < x then
 fact := 1
else
 fact := 2
end
```

- 测试结果

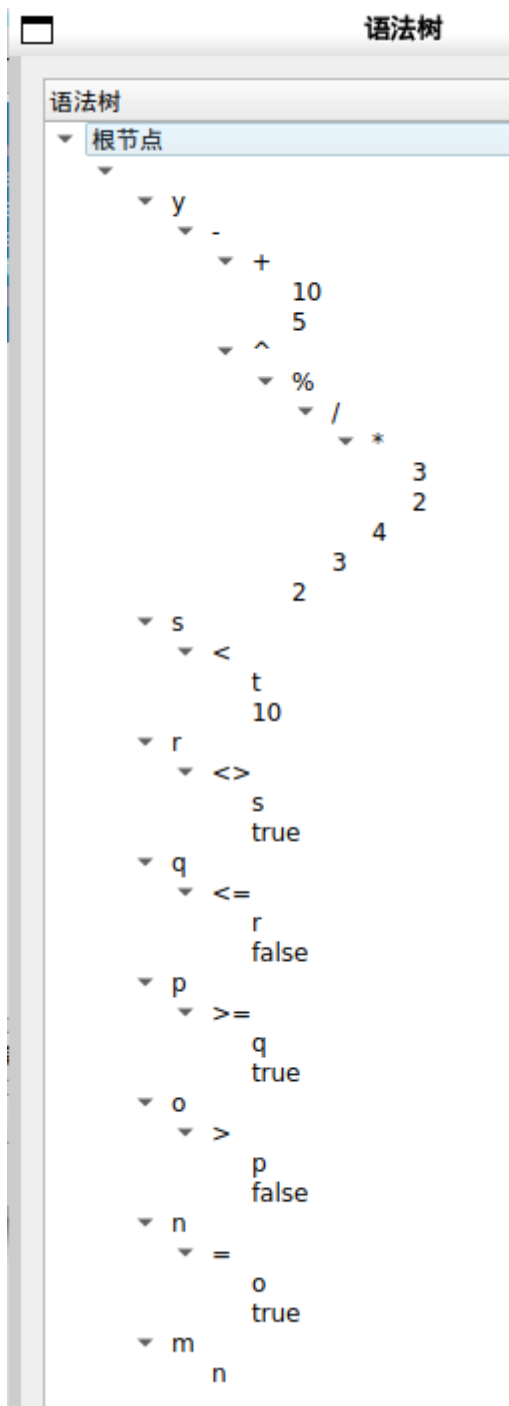


## 表达式语法树的生成

- 测试说明
  - 表达式测试
- 测试用源程序

- ```
y := 10 + 5 - 3 * 2 / 4 % 3 ^ 2;
s := t < 10; { less than }
r := s <> true; { not equal }
q := r <= false; { less than or equal }
p := q >= true; { greater than or equal }
o := p > false; { greater than }
n := o = true; { equal }
m := n { assignment }
```

- 测试结果



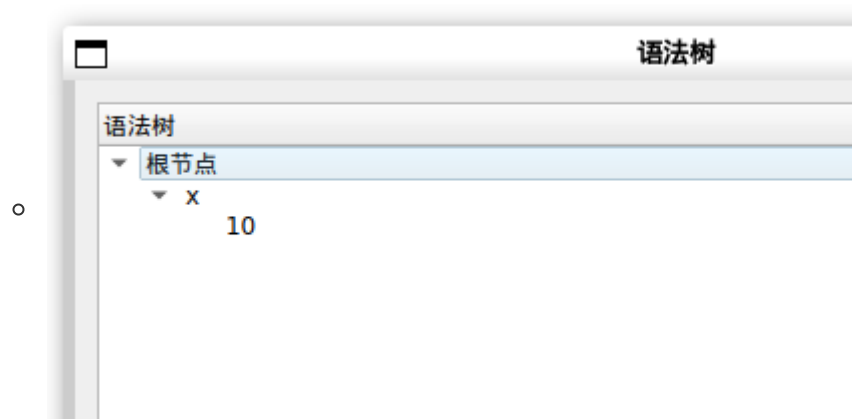
定义赋值语法树的生成

- 测试说明
 - 表达式测试
- 测试用源程序

```

{
    Test program if
}
x := 10
  
```

-
- 测试结果

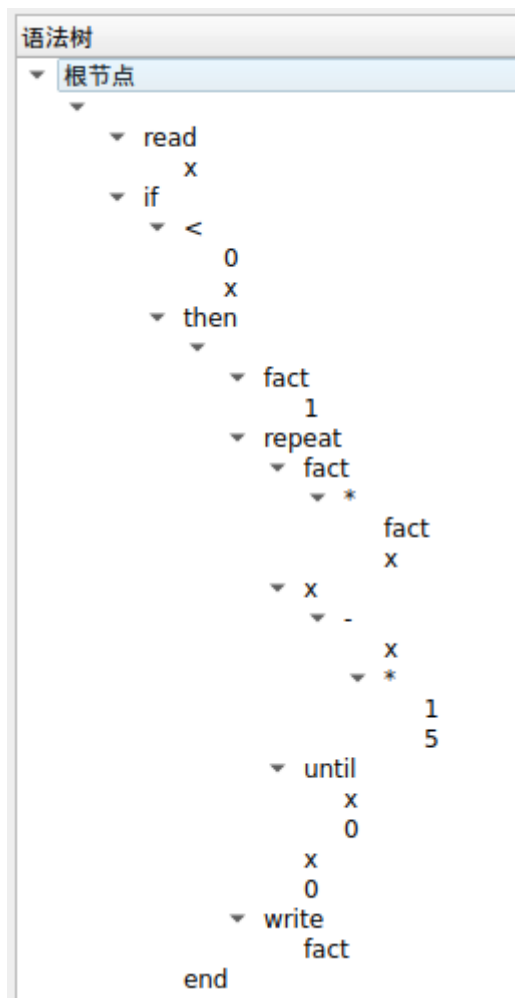


综合测试

- 测试说明
 - 综合程序设计
- 测试用源程序

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }
if 0 < x then { don't compute if x <= 0 }
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1 * 5
  until x = 0;
  write fact { output factorial of x }
end
```

-
- 测试结果



mini-c语言的测试

语法树修正代码

```

#include "../grammarAnalyzer/analyzer/analyzer.h"
#include <set>
#include <algorithm>
namespace std
{
    set<string> ignoreSigns = {"space", "tab", "enter", "(", ")",
    ";", "{", "}", "[", ""]};

    set<string> ops = {"+", "-", "*", "/", ">", "<", "^", "%", "<=",
    ">=", "=", "!="};
}
namespace std{
    extern "C" {
        void fixTree(Tree * tree){
            if (tree->children.empty()){
                return;
            }
            for (auto child : tree->children){
                fixTree(child);
            }
        }
    }
}

```

```

    }
    vector<Tree*> newChildren;
    // 对特定符号进行处理
    // exp
    if (tree->children.size() == 3 && ops.find(tree->children[1]->sign) != ops.end()) {
        tree->value = tree->children[1]->value;
        tree->children[1]->value = "";
    }

    // function
    if (tree->sign == "call") {
        tree->value = tree->children[0]->value;
        tree->children.erase(tree->children.begin());
    }
    // if
    if (tree->sign == "condition-stmt") {
        // save if and remove if from children
        tree->value = tree->children[0]->value;
        tree->children.erase(tree->children.begin());
        // find else
        int elseIndex = -1;
        for (int i = 0; i < tree->children.size(); i++) {
            if (tree->children[i]->sign == "else") {
                elseIndex = i;
                break;
            }
        }
        // if else exists
        if (elseIndex != -1) {
            tree->children[elseIndex]->children = tree->children[elseIndex+1]->children;

            tree->children.erase(tree->children.begin()+elseIndex+1);
        }
    }

    // return
    if (tree->sign == "return-stmt") {
        tree->value = tree->children[0]->value;
        tree->children.erase(tree->children.begin());
    }

    // while
    if (tree->sign == "while-stmt") {
        tree->value = tree->children[0]->value;
        tree->children.erase(tree->children.begin());
    }

```

```

    }

    // do-while
    if (tree->sign == "do-while-stmt") {
        tree->value = tree->children[1]->value;
        tree->children.erase(tree->children.begin()+1);

        int whileIndex = -1;
        for (int i = 0; i < tree->children.size(); i++) {
            if (tree->children[i]->sign == "while") {
                whileIndex = i;
                break;
            }
        }
        for (int i = 0; i < tree->children.size(); i++) {
            if (tree->children[i]->sign == "expression") {
                tree->children[whileIndex]-
>children.push_back(tree->children[i]);
                tree->children.erase(tree-
>children.begin()+i);

                break;
            }
        }
    }

    // expression
    if (tree->sign == "expression" && tree->children.size()
== 3) {

        tree->value = tree->children[0]->value; // name

        tree->children[0]->value = "";
        tree->children[1]->value = "";
        return;
    }

    // function definition
    if (tree->sign == "function-definition") {
        tree->sign += "+";
        tree->value = tree->children[1]->value;
        tree->children[1]->value = "";
    }

    // variable definition
    if (tree->sign == "variable-definition") {

        tree->value = tree->children[1]->value;
        tree->children[1]->value = "";
    }

    // 删除应该被忽略的符号

```

```

        for (auto child : tree->children) {
            if (ignoreSigns.find(child->sign) ==
ignoreSigns.end()) {
                newChildren.push_back(child);
            }
        }
        tree->children = newChildren;
        // 将只有一个子节点且节点值为空的符号合并
        if (tree->children.size() == 1 && tree->value == "") {
            tree->sign = tree->children[0]->sign;
            tree->value = tree->children[0]->value;
            tree->children = tree->children[0]->children;
        }
        // 删除value为空的符号
        newChildren.clear();
        for (auto child : tree->children) {
            if (!(child->value == "")) {
                newChildren.push_back(child);
            } else {
                for (auto c : child->children) {
                    newChildren.push_back(c);
                }
            }
        }
        tree->children = newChildren;
    }
}
}

```

词法分析器测试

- 测试说明
 - 所有token都在源程序中出现
- 测试用源程序

```

o  /*
    test tokens
    */

#include <stdio.h>

int main(void) {
    // Declare variables
    int x = 10;
    float y = 5.5;
    double z = 3.14;
}

```

```

// Check if x is greater than 0
if (x > 0) {
    int fact = 1;
    do{
        fact = fact * x;
        x = x - 1;
    }while (x > 0)

} else {
    print(x); // Output if x <= 0
}

// Additional operations to include all signs
int a = 10 + 5; // Addition
int b = a - 3; // Subtraction
int c = b * 2; // Multiplication
float d = c / 4.0; // Division
int e = (int)d % 3; // Modulus
int f = e ^ 2; // Bitwise XOR
int g = (f <= 10); // Less than or equal
int h = (g != 0); // Not equal
int i = (h >= 1); // Greater than or equal
int j = (i > 0); // Greater than
int k = (j == 1); // Equal
int l = k; // Assignment

// Array example
int arr[5];
arr[0] = arr[1] + arr[2]; // Array operations

// Return statement
return 0;
}

```

o

- 测试结果

o

```

1006 int
10 main
65 (
1005 void
66 )
69 {
1006 int
10 x
62 =
15 10
63 ;
1007 float

```

```
10 y
62 =
15 5.5
63 ;
1003 do
10 uble
10 z
62 =
15 3.14
63 ;
1000 if
65 (
10 x
59 >
15 0
66 )
69 {
1006 int
10 fact
62 =
15 1
63 ;
1003 do
69 {
10 fact
62 =
10 fact
52 *
10 x
63 ;
10 x
62 =
10 x
51 -
15 1
63 ;
70 }
1002 while
65 (
10 x
59 >
15 0
66 )
70 }
1001 else
69 {
10 print
65 (
10 x
```



```
66 )
63 ;
70 }
1006 int
10 a
62 =
15 10
50 +
15 5
63 ;
1006 int
10 b
62 =
10 a
51 -
15 3
63 ;
1006 int
10 c
62 =
10 b
52 *
15 2
63 ;
1007 float
10 d
62 =
10 c
53 /
15 4.0
63 ;
1006 int
10 e
62 =
65 (
1006 int
66 )
10 d
54 %
15 3
63 ;
1006 int
10 f
62 =
10 e
55 ^
15 2
63 ;
1006 int
```

```
10 g
62 =
65 (
10 f
56 <=
15 10
66 )
63 ;
1006 int
10 h
62 =
65 (
10 g
61 !=
15 0
66 )
63 ;
1006 int
10 i
62 =
65 (
10 h
58 >=
15 1
66 )
63 ;
1006 int
10 j
62 =
65 (
10 i
59 >
15 0
66 )
63 ;
1006 int
10 k
62 =
65 (
10 j
60 ==
15 1
66 )
63 ;
1006 int
10 l
62 =
10 k
63 ;
```

```

1006 int
10 arr
67 [
15 5
68 ]
63 ;
10 arr
67 [
15 0
68 ]
62 =
10 arr
67 [
15 1
68 ]
50 +
10 arr
67 [
15 2
68 ]
63 ;
1004 return
15 0
63 ;
70 }

```

◦

控制语句语法树的生成

1. if

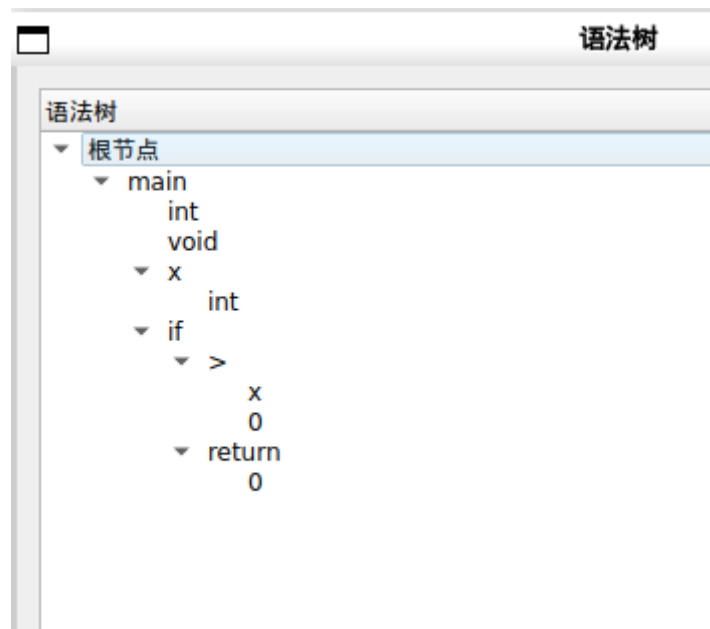
- 测试说明
 - 测试 if 语句
- 测试用源程序

```

// mini-c test
// single if
int main(void)
{
    int x;
    if (x > 0)
    {
        return 0;
    }
}

```

- 测试结果

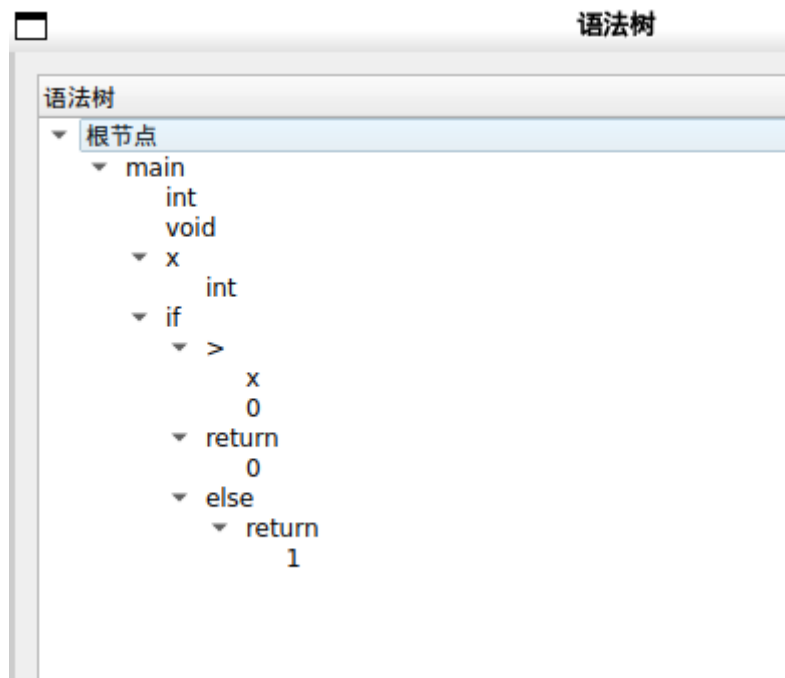


2. if-else

- 测试说明
 - 测试 if-else 语句
- 测试用源程序

```
// mini-c test
// if with else
int main(void)
{
    int x;
    if (x > 0)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}
```

- 测试结果



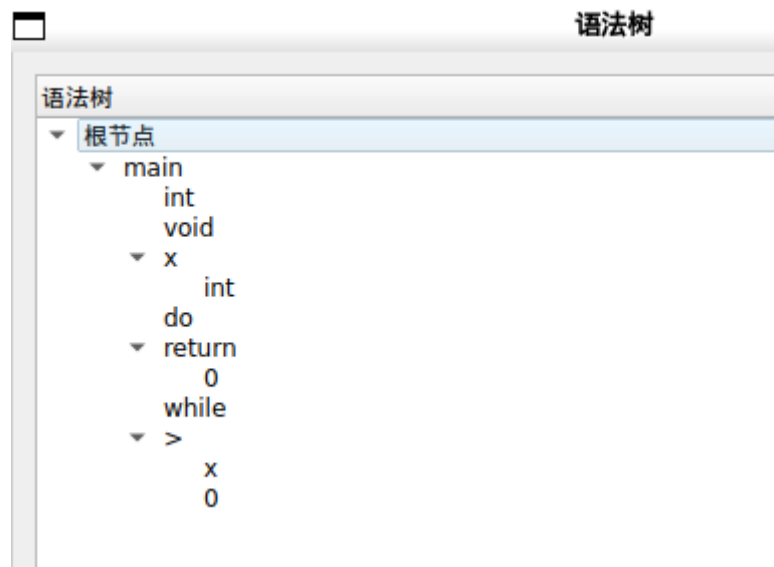
3. do-while

- 测试说明
 - 测试 do-while 语句
- 测试用源程序

```
// mini-c test
// do-while test

int main(void)
{
    int x;
    do
    {
        return 0;
    } while (x > 0);
}
```

- 测试结果



4. while

- 测试说明
 - 测试 while 语句
- 测试用源程序

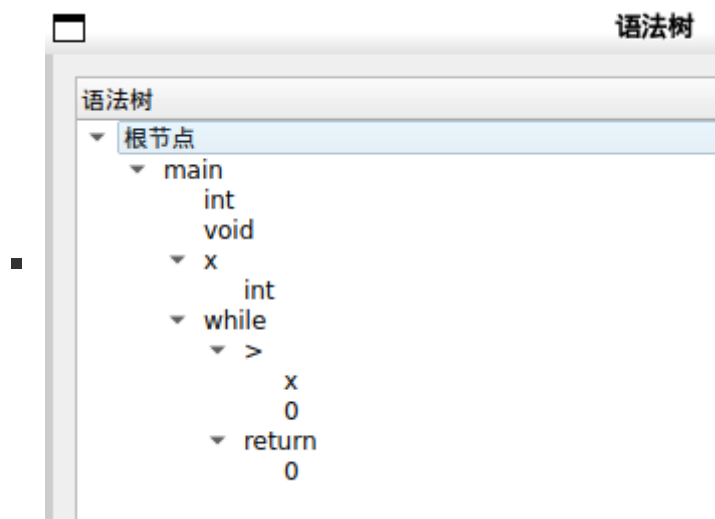
```

// mini-c test
// while test

int main(void)
{
    int x;
    while (x > 0)
    {
        return 0;
    }
}

```

- 测试结果



表达式语法树的生成

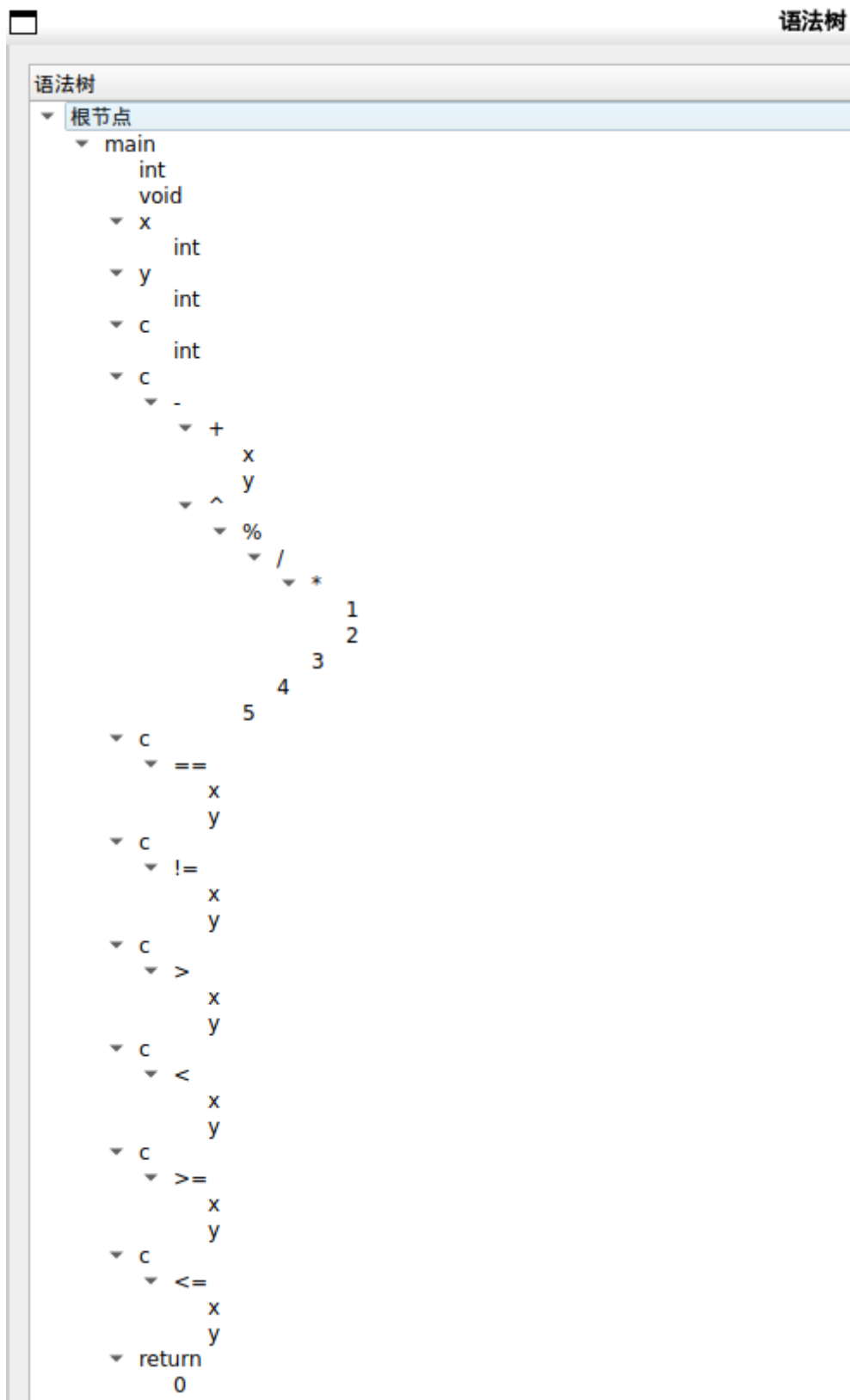
- 测试说明
 - 表达式测试
- 测试用源程序

- ```
// mini-c test
// op test

int main(void)
{
 int x;
 int y;
 int c;
 c = x + y - 1 * 2 / 3 % 4^5 ;
 c = x == y ;
 c = x != y ;
 c = x > y ;
 c = x < y ;
 c = x >= y ;
 c = x <= y ;

 return 0;
}
```

- 
- 测试结果



## 定义赋值语法树的生成

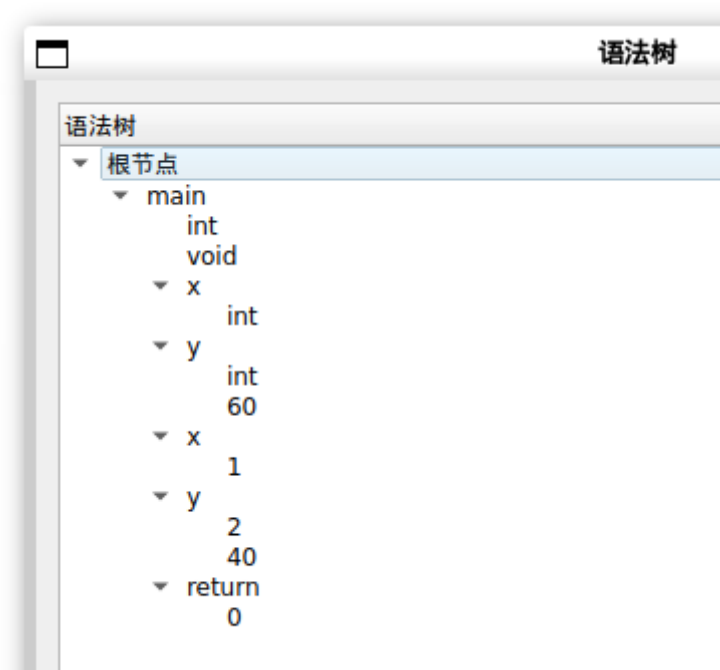
- 测试说明
  - 定义赋值语句
- 测试用源程序



- ```
// mini-c test
// assign test

int main(void)
{
    int x;
    int y[60];
    x = 1;
    y[2] = 40;
    return 0;
}
```

-
- 测试结果



综合测试

- 测试说明
 -
- 测试用源程序

- ```
// miniC示例程序：计算两个数的和与差

int main(void) {
 int a;
 int b; // 定义两个整型变量
 float result_add;
 float result_sub; // 定义两个浮点型变量用于存储结果

 // 赋值操作
 a = 10; // 第一个数
 b = 20; // 第二个数
```

```
// 加法和减法
result_add = a + b; // 计算和
result_sub = a - b; // 计算差

// 输出结果
print(result_add);

print(result_sub);

return 0; // 程序正常结束
}
```

- 
- 测试结果

